

# Newton School of Technology

Capstone Project — System Design

---

## System Design & Architecture

### Final Project Report

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| Project Title   | SmartCampus — Campus Management System                          |
| Domain / Sector | Education Technology (EdTech)                                   |
| Team Members    | Vaibhav Rajput, Parv Goyal, Kanchan Rani, Vaibhav, Anurag Kumar |
| Institute       | Newton School of Technology                                     |
| GitHub Repo     | <a href="#">SmartCampus on GitHub</a>                           |
| Live Demo       | <a href="#">SmartCampus Live Demo</a>                           |

## 2. Executive Summary

- **Problem:** Campus operations — attendance, grade management, assignment submission, and user administration — are fragmented across manual processes, causing inefficiencies for administrators, faculty, and students.
- **Approach:** SmartCampus is a modular TypeScript backend (Node.js + Express) with a React frontend, backed by PostgreSQL (Neon), secured via JWT authentication, and structured using layered MVC with Repository, Strategy, and Factory patterns.
- **Key Design Decisions:** Layered MVC with Repository pattern (SQL separated from business logic); Strategy + Factory pattern for role-based dashboards; JWT with role middleware; Neon PostgreSQL for cloud data persistence.
- **Key Outcomes:** Fully functional multi-role system (Admin, Faculty, Student) covering users, courses, enrollments, attendance, assignments, grades, dashboards, and notifications. API-driven, mobile-responsive UI with light/dark theme.

## 3. Domain & Business Context

- **Domain Overview:** Education Technology: fragmented tools for user management, attendance, grading, and communication. Centralisation reduces overhead and improves visibility across all roles.
- **Primary Stakeholder:** Administrators (system-wide management), Faculty (course, attendance, grading), Students (enrollment, submissions, performance).
- **Problem Selection:** Manual academic workflows create errors and delays. A unified system improves operational efficiency and data accuracy.
- **Definition of Success:** All three roles complete core workflows end-to-end within a single system.

## 4. Problem Statement & Objectives

### 4.1 Formal Problem Statement

Campus stakeholders rely on fragmented, manual tools for academic management, creating data inconsistency, delayed reporting, and high operational overhead. SmartCampus provides a single role-driven system for all core academic workflows.

### 4.2 Project Scope

- **In Scope:** User auth and RBAC (Admin, Faculty, Student); course creation; enrollment; session-based attendance; assignment upload (Multipart); grade management; role-based dashboards; notifications. Stack: Node.js, Express, TypeScript, PostgreSQL (Neon), React, Vite, JWT.
- **Out of Scope:** Payments, video streaming, live scheduling, mobile native apps, third-party LMS integrations.

### 4.3 Success Criteria

- All roles perform core workflows end-to-end
- API response time < 200ms for 95% of requests under normal load
- Zero critical vulnerabilities in authentication flows
- Session-based attendance enables accurate per-student percentage calculation

## 5. Requirements Analysis

## 5.1 Functional Requirements

- **Auth:** Registration, login with JWT, RBAC middleware on all protected routes
- **Users:** Admin CRUD across all roles
- **Courses:** Faculty/Admin create and manage courses
- **Enrollments:** Student enrollment; Admin manages
- **Attendance:** Faculty records sessions; student percentage auto-calculated
- **Assignments:** Faculty creates; students upload files via Multer
- **Grades:** Faculty assigns; students view performance
- **Dashboard:** Role-specific data via Strategy-Factory pattern
- **Notifications:** Decoupled via NotificationPublisher interface

## 5.2 Non-Functional Requirements

- **Performance:** API responses under 200ms for standard CRUD
- **Scalability:** Neon PostgreSQL connection pooling (max 5); modular services allow independent scaling
- **Security:** JWT auth, bcrypt hashing, role middleware, CORS policy
- **Maintainability:** TypeScript throughout; SRP enforced across controllers, services, and repositories

## 5.3 Constraints & Assumptions

- Neon free tier limits DB connections to 5
- Assignment files stored locally under server/uploads/ (no cloud storage in v1)
- Assumes stable internet access for Neon cloud DB connection

# 6. System Architecture

## 6.1 Architecture Style & Justification

Layered MVC monolithic architecture with clear service and repository separation. Chosen for team size (5), academic timeline, and straightforward single-server deployment. Microservices would add unnecessary operational complexity.

## 6.2 High-Level Architecture

- **Presentation Tier:** React + Vite (port 5173) — role-based navigation and conditional rendering per JWT role
- **Application Tier:** Node.js + Express + TypeScript (port 4000) — controllers / services / repositories
- **Data Tier:** PostgreSQL on Neon; schema.sql; dev\_seed.sql
- **External Services:** Neon (cloud PostgreSQL), Multer (file uploads), JWT (auth)

## 6.3 Component Breakdown

- **AuthService:** Login, registration, token generation. Depends on UserRepository and TokenProvider interface.
- **DashboardService:** Delegates to DashboardStrategyFactory → selects Admin/Faculty/Student strategy.
- **AttendanceService:** Session-based records; calculates per-student percentage.
- **AssignmentService:** Assignment creation and submission handling with Multer file integration.
- **NotificationService:** Publishes and retrieves notifications via NotificationPublisher interface.

## 6.4 Key Data Flows

- **Login:** POST /api/auth/login → AuthController → AuthService → UserRepository (DB) → JWT returned

- **Attendance:** POST /api/attendance → AttendanceService → AttendanceRepository → percentage recalculated on GET
- **Dashboard:** GET /api/dashboard → DashboardService → StrategyFactory (role from JWT) → role-specific data

## 6.5 Deployment

- Backend: port 10000 | Frontend: Vite port 5173 | DB: Neon PostgreSQL (SSL) | Files: server/uploads/
- No containerisation in v1; Docker and CI/CD deferred to future scope

## 7. Database Design

### 7.1 Selection & Justification

PostgreSQL selected for relational integrity, complex join support (enrollments, grades), and Neon serverless compatibility.

### 7.2 Core Entities

Users, Courses, Enrollments, AttendanceSessions, AttendanceRecords, Assignments, Submissions, Grades, Notifications. Key relationships: Users ↔ Enrollments ↔ Courses (M:M); Courses ↔ Assignments (1:M); Assignments ↔ Submissions (1:M).

### 7.3 Schema & Integrity

- **Key Tables:** users (id, name, email, password\_hash, role), courses (id, title, faculty\_id), enrollments (student\_id, course\_id), attendance\_sessions, attendance\_records, assignments, submissions (file\_path), grades, notifications (is\_read)
- **Constraints:** FK across all relational tables; UNIQUE on enrollments(student\_id, course\_id); NOT NULL on required fields; role CHECK (admin/faculty/student); CASCADE DELETE where appropriate
- **Indexing:** users.email (unique/login); enrollments(student\_id, course\_id); attendance\_records(session\_id, student\_id); notifications(user\_id, is\_read)

## 8. API Design

### 8.1 Style & Auth

RESTful over HTTP. Plural noun URLs. JWT Bearer tokens (7d expiry); role extracted from payload; RBAC enforced per route. Consistent error schema: { error, message }.

### 8.2 Endpoint Reference

- **Auth:** POST /api/auth/login, POST /api/auth/register
- **Users:** GET / POST / PUT / DELETE /api/users/:id (Admin)
- **Courses:** GET / POST / PUT / DELETE /api/courses
- **Enrollments:** GET / POST / DELETE /api/enrollments
- **Attendance:** POST /api/attendance | GET /api/attendance/percentage/:studentId
- **Assignments:** GET / POST /api/assignments | POST /api/assignments/:id/submit
- **Grades:** GET / POST / PUT /api/grades
- **Dashboard:** GET /api/dashboard (role-specific response)
- **Notifications:** GET / POST / PATCH /api/notifications/:id/read

## 9. Design Patterns & Engineering Principles

### 9.1 Design Patterns Applied

- **Repository Pattern:** All DB access in repository classes; services never write raw SQL. BaseRepository.ts → extended by UserRepository, CourseRepository, etc. — src/repositories/
- **Strategy Pattern:** Role-based dashboard logic in separate Admin/Faculty/Student strategy classes, each implementing DashboardStrategy. New roles added without touching existing code. — src/strategies/DashboardStrategies.ts
- **Factory Pattern:** DashboardStrategyFactory.ts selects correct strategy at runtime from JWT role. Controllers fully decoupled from concrete types. — src/factories/DashboardStrategyFactory.ts

### 9.2 OOP Principles

- **Encapsulation:** SQL and business rules hidden inside repository/service classes (UserRepository.ts, AuthService.ts)
- **Inheritance:** BaseRepository.ts holds shared DB behaviour; all repositories extend it
- **Polymorphism:** DashboardStrategy interface allows Admin/Faculty/Student instances to be used interchangeably
- **Abstraction:** Interfaces (TokenProvider, NotificationPublisher, DashboardStrategy) hide implementation behind clean contracts

### 9.3 SOLID Principles

- **SRP:** Controllers handle HTTP only; services hold business logic; repositories handle persistence
- **OCP:** New dashboard roles added by creating a new strategy class — no changes to DashboardService
- **LSP:** Any repository extending BaseRepository and any strategy implementing DashboardStrategy is substitutable
- **ISP:** Small interfaces (TokenProvider, NotificationPublisher, DashboardStrategy) — no forced unused methods
- **DIP:** DashboardService depends on DashboardStrategy abstraction; AuthService depends on TokenProvider interface

## 10. Security Design

- **Authentication:** JWT Bearer tokens; token contains user ID and role; validated by auth middleware on every protected route
- **Authorisation:** RBAC enforced at route middleware level before reaching controllers
- **Password Security:** bcrypt with salting; plaintext passwords never stored or logged
- **Injection Prev.:** Parameterised queries via PostgreSQL driver; Repository pattern prevents raw SQL string interpolation
- **CORS:** CLIENT\_URL env variable whitelists frontend origin
- **File Uploads:** Multer validates uploads; files stored in server/uploads/submissions/ with server-generated names
- **Secrets:** All secrets (JWT\_SECRET, DATABASE\_URL) in server/.env; no hardcoded credentials
- **Transport:** DATABASE\_SSL=true enforces SSL for Neon; HTTPS required for production

## 11. Frontend Architecture

- **Framework:** React + Vite. Fast HMR, minimal config overhead. <http://localhost:5173>.

- **Structure:** Login → Role Dashboard (Admin/Faculty/Student) → Module pages. JWT role drives route access and conditional rendering.
- **UI/UX:** Light/dark CSS theme toggle; modern minimalist layout; mobile-responsive
- **Role-Based UI:** Admin: full user and course management | Faculty: courses, attendance, assignments, grades | Student: enrollment, submissions, performance
- **Screenshots:** See Appendix: Login, Admin Dashboard, Faculty Dashboard, Student Dashboard, Attendance, Assignment, Grade views

## 12. Testing Strategy

- **Approach:** Manual API testing via Postman for all endpoint groups; seed data scenarios verify multi-role workflows
- **Integration Testing:** Auth → Middleware → Controller → Service → Repository flow verified for each module
- **API Testing:** All endpoints: happy path, 401 (missing auth), 403 (wrong role), 400 (invalid input)
- **Security Testing:** Token bypass, role escalation, missing token scenarios all verified
- **Test Cases:** See Appendix: Login, Enrollment, Attendance, Submission, Grade test cases with input/expected/actual/pass-fail

## 13. Insights & Learnings

- JWT stateless tokens reduce session storage overhead but require careful expiry management to avoid security gaps.
- Repository pattern early prevented service-layer bloat and made unit testing significantly easier as the project scaled.
- Strategy + Factory for dashboards enabled role-specific logic to be developed in parallel without merge conflicts.
- Session-based attendance (recording sessions, not marks) enabled accurate percentage calculation without retroactive recomputation.
- Neon connection pooling (max 5) requires careful async handling; unresolved promises caused connection exhaustion early on.
- TypeScript strict typing caught several null-reference bugs at compile time that would have been runtime errors in plain JS.

## 14. Recommendations

- **1:** Rate limiting on /api/auth/login → prevents brute-force attacks → Low effort (express-rate-limit)
- **2:** Cloud file storage (S3 / Cloudinary) → eliminates local disk dependency; enables horizontal scaling → Medium effort
- **3:** WebSocket real-time notifications (Socket.io) → improves UX for live updates → Medium effort
- **4:** Automated test suite (Jest + Supertest) → reduces regression risk → Medium effort, high long-term value
- **5:** Docker containerisation → reproducible deployment; eliminates environment-specific setup issues → Low-medium effort

## 15. Limitations

- Local file storage not suitable for multi-instance/cloud deployment in current form

- No rate limiting on auth endpoints — brute-force risk in production
- Neon free tier: 5 connection cap insufficient for high concurrency
- No automated test suite; regression risk grows with codebase
- No real-time updates; notifications require page refresh
- JWT tokens cannot be invalidated before expiry (no blacklist / refresh token)

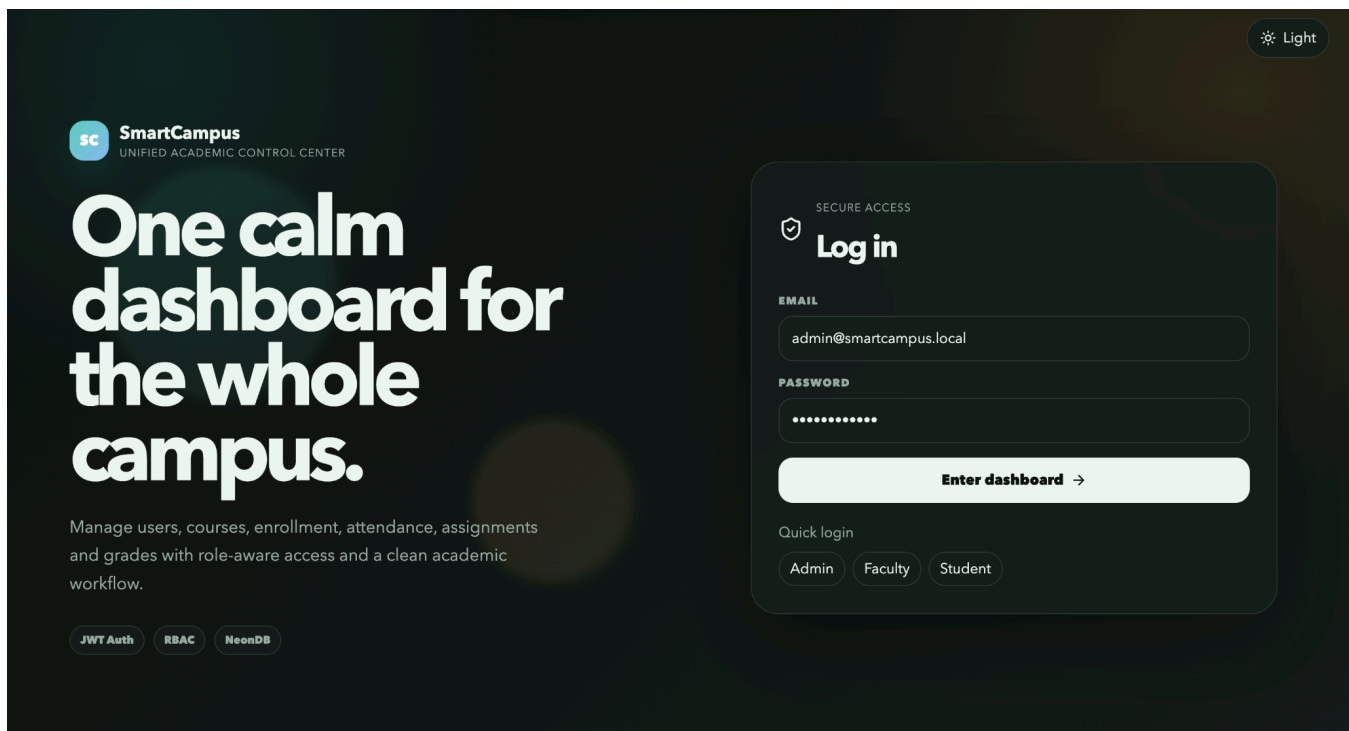
## 16. Future Scope

- Real-time notifications and live attendance via WebSockets (Socket.io)
- Cloud file storage migration (AWS S3 or Cloudinary)
- Docker + CI/CD pipeline (GitHub Actions)
- Monitoring: Prometheus + Grafana for API performance dashboards
- Refresh token mechanism for secure long-lived sessions
- AI analytics: grade trend prediction and attendance risk alerts for faculty

## 17. Conclusion

SmartCampus addresses the fragmentation of campus academic management by delivering a unified, role-driven system for administrators, faculty, and students. A layered MVC architecture with Repository, Strategy, and Factory patterns was chosen to enforce separation of concerns without over-engineering. The most critical decision — decoupling dashboard logic via Strategy + Factory — enabled parallel development and clean role isolation. The system covers user management, courses, enrollment, session-based attendance, assignment submission, grading, and notifications, all secured by JWT RBAC. Recommended next step: cloud file storage migration and rate limiting for production readiness.

## 18. Appendix



SC

SmartCampus

ACADEMIC OS

Overview

Users

Courses

Light mode

Logout

WELCOME BACK

System Admin

ADMIN

admin@smartcampus.local

ADMIN CONSOLE

Campus command center

A clean overview of users, courses, enrollments and academic activity.

Total users

6

Courses

1

Enrollments

2

Assignments

1

Users by role

Live from NeonDB

faculty

1

admin

2

student

3

Recent enrollments

akhilesh

CS101

Riya Sharma

CS101

SC

SmartCampus

ACADEMIC OS

Overview

Attendance

Assignments

Light mode

Logout

WELCOME BACK

Dr. Meera Singh

FACULTY

faculty@smartcampus.local

FACULTY WORKSPACE

Teach, track, and grade

Manage your assigned courses, attendance sessions, and student submissions.

Assigned courses

1

Pending grades

0

Recent sessions

1

Assigned courses

Faculty only

CS101

Introduction to Computing



SC

SmartCampus

ACADEMIC OS

Overview

Courses

Performance

Notifications

Light mode

Logout

WELCOME BACK

Riya Sharma

STUDENT

student@smartcampus.local

STUDENT PORTAL

Your academic snapshot

Track enrolled courses, grades, attendance and updates from one focused place.

Enrolled courses

1

Published grades

1

Attendance

100%

My courses

Student view

CS101

Introduction to Computing

SC

SmartCampus

ACADEMIC OS

Overview

Attendance

Assignments

Light mode

Logout

WELCOME BACK

Dr. Meera Singh

FACULTY

faculty@smartcampus.local

ATTENDANCE

Create sessions and mark presence

Attendance is stored per class session, so percentages stay accurate.

Create session

COURSE

Select course

DATE

dd/mm/yyyy

TOPIC

Create session

Mark attendance

SESSION ID

STUDENT ID

Mark present

SC

SmartCampus

ACADEMIC OS

Overview

Courses

Performance

Notifications

Light mode

Logout

WELCOME BACK

Riya Sharma

STUDENT

student@smartcampus.local

COURSES

Enroll and submit coursework

Browse active courses and submit files for assignments posted by faculty.

Active courses1

CS101 - Introduction to ComputingEnrolled

Assignments1

Smoke Test Assignment 1776096178994Submitted

CS101 - Due Jan 1, 2027

SC

SmartCampus

ACADEMIC OS

Overview

Attendance

Assignments

Light mode

Logout

WELCOME BACK

Dr. Meera Singh

FACULTY

faculty@smartcampus.local

ASSIGNMENTS

Post work and review submissions

Create course assignments, inspect submissions, and publish grades.

Create assignment

COURSE

Select course

TITLE

DEADLINE

dd/mm/yyyy, --:-- --

MAX MARKS

10

Create assignment

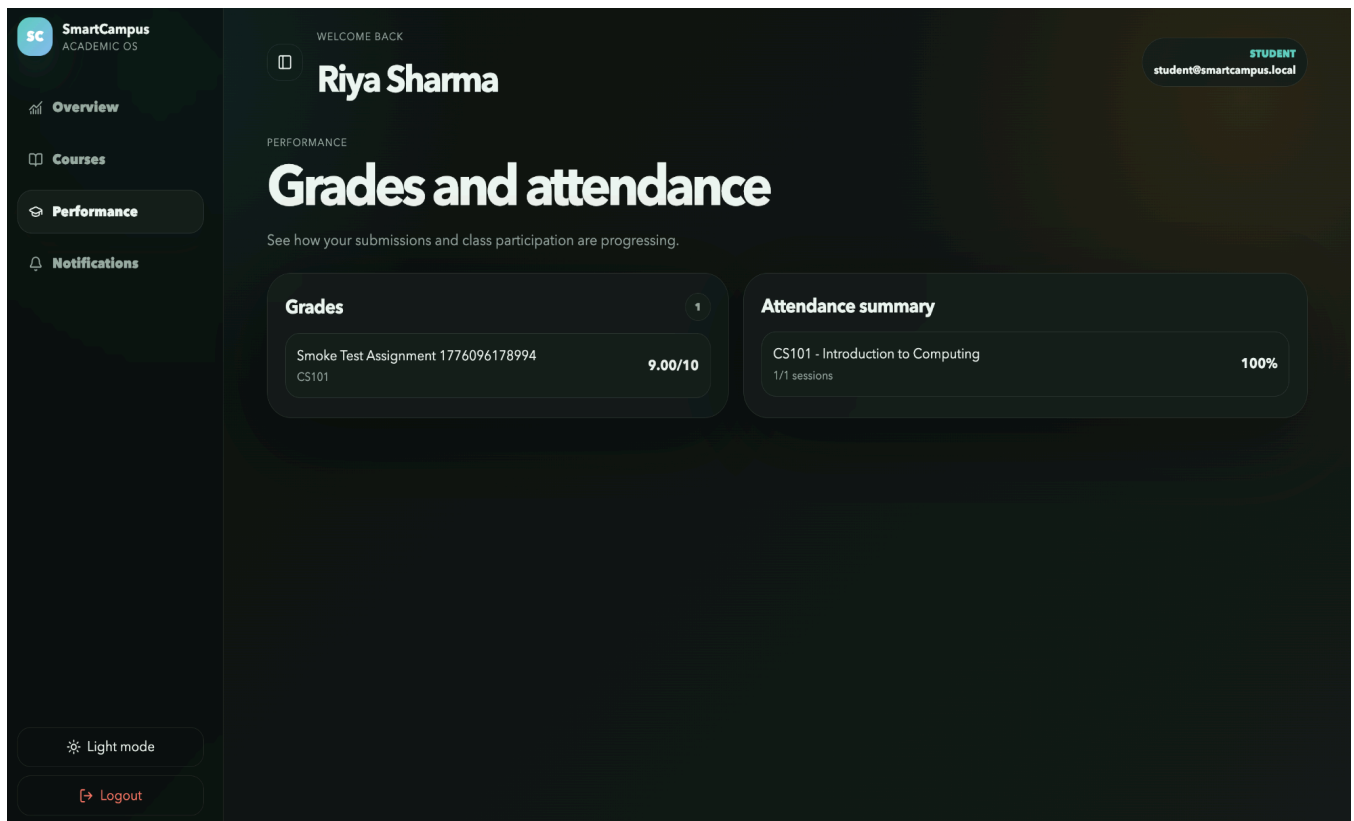
Assignments1

Smoke Test Assignment 1776096178994CS101

Submissions0 loaded

No submissions loaded

Select an assignment to review submissions.



## 19. Contribution Matrix

| Team Member    | Role              | System Design | Implementation | Testing | Arch Diagrams | Report | PPT & Viva |
|----------------|-------------------|---------------|----------------|---------|---------------|--------|------------|
| Vaibhav Rajput | Frontend Dev      |               | ✓              | ✓       |               |        | ✓          |
| Parv Goyal     | Backend Lead      |               | ✓              |         |               |        | ✓          |
| Kanchan Rani   | DB Lead           | ✓             | ✓              |         |               | ✓      | ✓          |
| Vaibhav        | Academic APIs     |               | ✓              |         |               |        | ✓          |
| Anurag Kumar   | Docs & Sys Design | ✓             | ✓              |         | ✓             |        | ✓          |